

ALG Exam 2025

Kevin Alex Nielsen - Studienummer: 20244703

Question 1

1.1

a) TRUE, b) FALSE, c) FALSE, d) FALSE, e) FALSE

1.2

a) FALSE, b) TRUE, c) TRUE, d) TRUE, e) TRUE

1.3

a) TRUE, b) TRUE, c) TRUE, d) FALSE, e) FALSE

Question 2

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ 7T(n/3) + n^2 & \text{if } n > 1 \end{cases}$$

$$a = 7, \quad b = 3, \quad f(n) = n^2, \quad n^{\log_b a} = n^{\log_3 7} \approx n^{1.77}$$

Since $f(n)$ is larger than $n^{\log_b a}$, we will use case 3 of the master theorem.

We will check if $f(n) = \Omega(n^{\log_b a + \epsilon})$ for any $\epsilon > 0$. If we use an $\epsilon < (2 - 1.77) \Rightarrow \epsilon < 0.23$, this case holds. Therefore, we can deduce from the master theorem that the asymptotic runtime of $T(n) = \Theta(f(n)) = \Theta(n^2)$.

Question 3

3.1

a) FALSE, b) TRUE, c) TRUE, d) FALSE, e) TRUE

3.2

a)

The root of the proper binary max-heap is at index 0.

b)

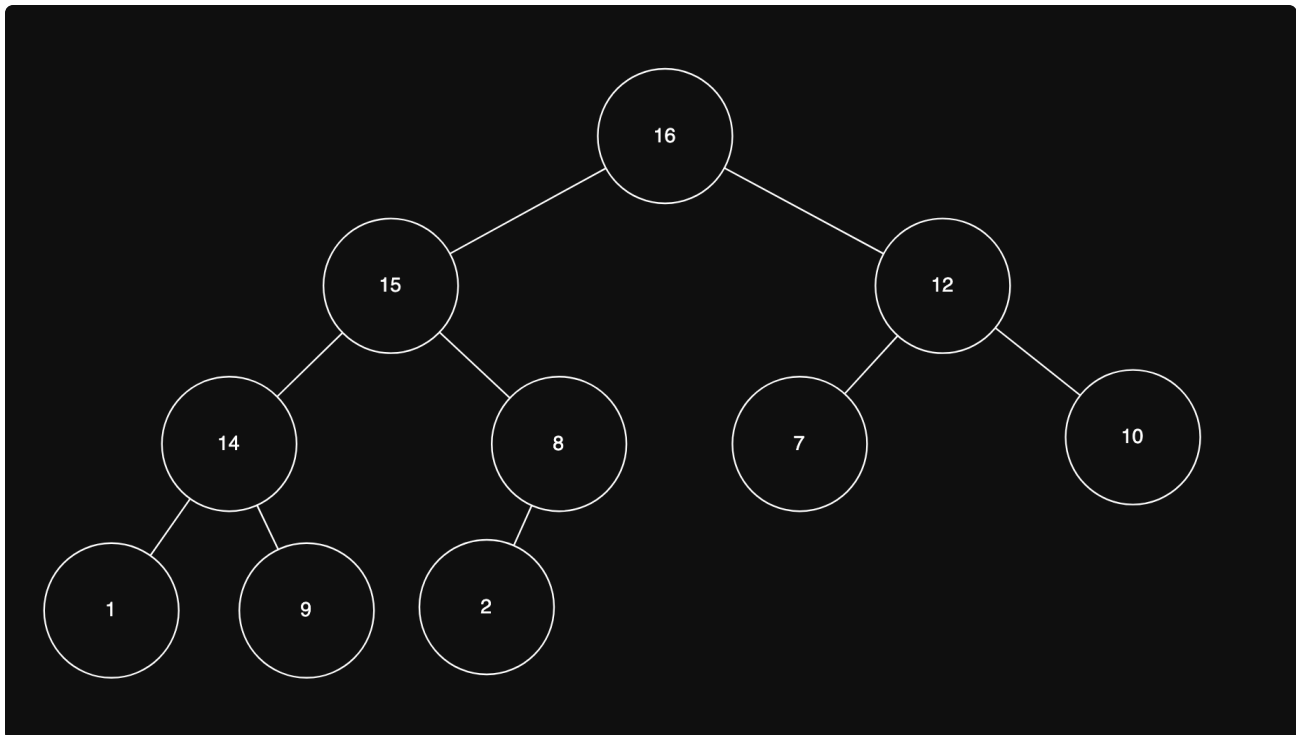
Max-Heapify is a recursive algorithm, that will takes in an array A and an index i where $\text{Left}(i)$ and $\text{Right}(i)$ are max-heaps but $A[i]$ might be less than $\text{Left}(i)$ and $\text{Right}(i)$.

With the current state of A , $A[2]$ is not a Max-Heap, but $A[4]$ and $A[5]$ are. This means that we need to call Max-Heapify on index 2. This results in a recursive call to run Max-Heapify on index 4, since it is no longer a Max-Heap since $14 > 9$.

$A[3]$ is not a valid subtree, but index 6 and 7 are leafs. This means we need to call Max-Heapify on index 3 as well. Now, A is a valid Max-Heap.

Final answer: We need to call Max-Heapify on indices 2, 4 and 3.

c)



3.3

a)

The first call to Partition is with a pivot position of index 8 with a value of "s". This returns 5 ($i + 1$). This means that we call Quicksort recursively with the parameters $p = 1$, $r = 4$,

which then calls partition (the second time) with these same parameters, resulting in the array $A = [e, a, a, f, u, t, u, s]$.

b)

With the best case scenario, quicksort will create 2 new sub-problems of size $n/2$ with each recursive call. This leads to a runtime of $\Theta(n \log n)$. The merge sort algorithm also has a asymptotic runtime of $\Theta(n \log n)$, meaning that quicksort is not asymptotically faster in the best case scenario.

3.4

a)

$h(k) = m(k * 0.91 - \lfloor k * 0.91 \rfloor)$ with $m = 8$.

$$\begin{aligned} h(9) &= 8(9 * 0.91 - \lfloor 9 * 0.91 \rfloor) = 8(8.19 - \lfloor 8.19 \rfloor) = 8(0.19) &= 1.52 \\ h(3) &= 8(3 * 0.91 - \lfloor 3 * 0.91 \rfloor) = 8(2.73 - 2) = 8(0.73) &= 5.84 \\ h(6) &= 8(6 * 0.91 - \lfloor 6 * 0.91 \rfloor) = 8(0.46) &= 3.68 \\ h(8) &= 8(8 * 0.91 - \lfloor 8 * 0.91 \rfloor) = 8(0.28) &= 2.24 \\ h(4) &= 8(4 * 0.91 - \lfloor 4 * 0.91 \rfloor) = 8(0.64) &= 5.12 \\ h(7) &= 8(7 * 0.91 - \lfloor 7 * 0.91 \rfloor) = 8(0.37) &= 2.96 \end{aligned}$$

There are no collisions with these entries. Using chaining, this would mean that for each of the keys, there would be a linked list with only a single value as displayed above. If there were collisions, then they would be handled by adding another element to a linked list on that spot in the hash table.

b)

For double hashing with linear probing, we have $h(k, i) = (h_1(k) + ih_2(k)) \mod m$.

Our auxiliary hash functions are $h_1(k) = \lfloor k^2/5 \rfloor$ and $h_2(k) = 4k + 1$.

During the first check, we start of with $i = 0$. We move to spot $T[h_1(k) \mod m]$, and with $k = 5$ and $m = 8$, we have

$$T[h_1(k) \mod m] = T[\lfloor 5^2/5 \rfloor \mod 8] = T[5 \mod 8] = T[5]$$

If no value was found in this spot, we would increment i by 1 and compute again, so with $i = 1, k = 5, m = 8$ we have:

$$\begin{aligned} T[(h_1(k) + ih_2(k)) \mod m] &= T[(\lfloor 5^2/5 \rfloor + (4(5) + 1)) \mod 8] \\ &= T[(26) \mod 8] \\ &= T[2] \end{aligned}$$

Question 4

4.1

The best case input for WeirdSort would be an array that is sorted from high to low. The while-loop on line 3 runs while `A.size > 0`, meaning that `A` needs to be emptied for the loop to finish. On each iteration of the for-loop on line 4, `A` gets reduced if either `S.size == 0` or `A[i] ≤ S[S.size]`.

If `A` is sorted from high to low, that means that every iteration of the for-loop will reduce the size of `A` by 1. This means that after the for-loop is done running, `A.size == 0`. The lowest value will be pushed to `B` and second while-loop will empty `S` into `B`. In this way, the asymptotic runtime will be $\Theta(n)$, since we have iterated through n exactly twice.

The worst case scenario will be that `A` is sorted from low to high. In this way, every time the for-loop is done running, it will only have decreased the size of `A` by 1. This means the first while-loop has to run n times and for each time it has to run the for loop 1 less time. This gives us a final asymptotic runtime of $\Theta(n \log n)$.

4.2

To show that the WeirdSort algorithm sorts the algorithm as expected, we could state a loop invariant like this: "At the end of each iteration of the first while-loop, the array `B` consists of a inversely sorted list."

Initialization:

`B` is empty and trivially sorted.

Maintenance:

The for-loop compares all values of `A` with the latest element in `S`, looking for the smallest value in `A`. After the for-loop is done iterating, all values smaller than the last element of `A` will have been added to `S` in a sorted order.

Question 5

5.1

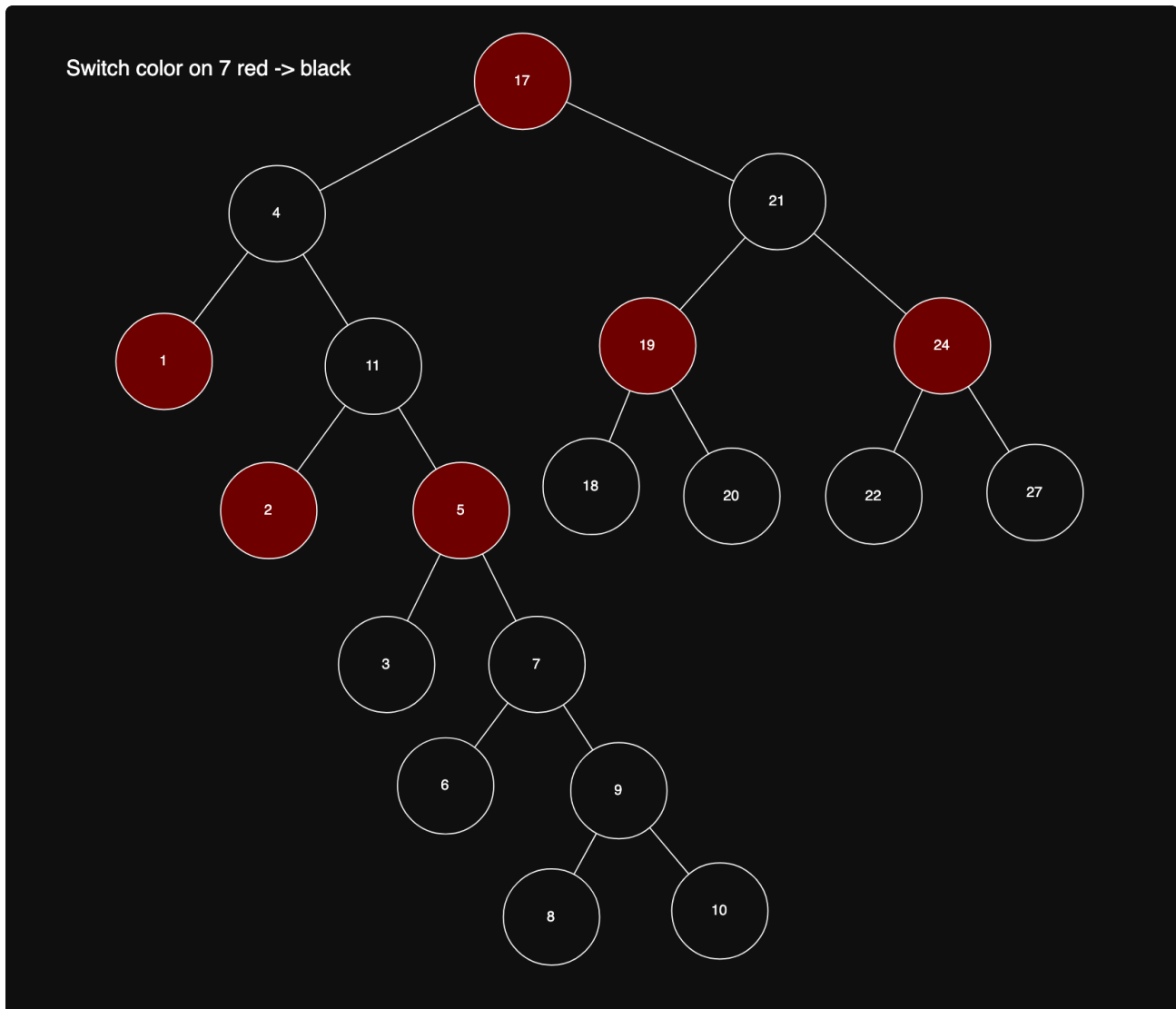
T violates the following red-black properties:

- "The root is black": The root (17) is red.

- "If a node is red, then its children are black": 7 is red, but 5 is also red.
- "All simple paths from a node to a leaf in one of its subtrees contain the same amount of black nodes": From 7 to 3, there is one black node, but from 7 to 10, there are two black nodes.

5.2

If we perform a Right-Rotate on 7, and then a Right-Rotate on 11, and switch the color on 7 from red to black, we get a tree that looks like the following



Question 6

6.1

This is the single destination shortest-paths problem. It can be solved using the Bellman-Ford algorithm, which has a asymptotic runtime of $O(VE)$.

6.2

This could be done by running the Bellman-Ford algorithm one time for each of the guests with the weights of all edges updated accordingly. In this way, we get a weighted shortest path from each guest to the host, and the added "scores" for each guest would not interfere with the scores of the other guests.

6.3

To find out what friend lives the closest to everyone else, we could run the Floyd-Warshall algorithm to get the distances from any guest to another. Then, for each guest we would check the total distance to all other guests, and then compare the totals.

With that in mind, the algorithm could look like the following:

```
Find-Closest-Guest()
1  distances = Floyd-Warshall()
2  lowest_distance = infinity
3  for person in distances:
4      total_distance = 0
5      for guest in person:
6          total_distance = total_distance + guest
7
8      if total_distance < lowest_distance:
9          lowest_distance = total_distance
```

The Floyd-Warshall algorithm runs in an asymptotic time of $O(n^3)$.

For each person, we iterate through all other people, meaning that the summing and comparing will run in asymptotic time of $O(n^2)$, meaning that the total asymptotic run time of our algorithm is $O(n^3)$.